

# String similarity learning notes

## Round 7

**离线字符串匹配 (offline string matching)**：text  $T$  已知时，先给  $T$  建索引，之后多次查询 pattern 会更快。核心结构：Suffix Trie / Suffix Tree / Suffix Array + LCP 优化（用于搜索引擎、DNA 匹配、全文检索）。

### 1. Online vs Offline

- Online（在线）： $T$  边到边匹配（KMP、暴力等），至少扫一遍 text，时间下界接近  $O(n)$ 。
- Offline（离线）： $T$  事先已知，先建立 index，再查多个 pattern。

### 2. Suffix Trie (后缀 Trie)

做法：把所有后缀  $T[i..n-1]$  都插入 Trie。例  $T = \text{"inning"}$  的后缀：inning / nning / ning / ing / ng / g。

查询：沿着 pattern  $P$  在 Trie 里走；若能走完，说明  $P$  是某个后缀的前缀  $\Rightarrow P$  在  $T$  中出现；最后输出所有 occurrence。

复杂度：查询  $O(m + \text{occ})$ ， $m = |P|$ ， $\text{occ}$  = 出现次数。

缺点：空间爆炸：总字符数  $n + (n-1) + \dots + 1 = n(n+1)/2 \Rightarrow$  空间/建树时间都是  $O(n^2)$ 。

### 3. Suffix Tree (后缀树)

核心改进：**路径压缩**（把 Trie 中“无分叉的一长段”压成一条边）。边不存字符串本身，只存区间  $(i, j)$ ，表示对应  $T[i..j]$ 。

为什么空间  $O(n)$ ：除 root 外，每个内部节点至少 2 个孩子；叶子约  $n$  个，分叉节点也是  $O(n) \Rightarrow$  总节点数  $O(n)$ 。

结论（经典）：建树  $O(n)$ ，查询  $O(m + \text{occ})$ ，空间  $O(n)$ （常数较大、实现难）。

| 结构          | 建      | 查                   | 空间     |
|-------------|--------|---------------------|--------|
| Suffix Tree | $O(n)$ | $O(m + \text{occ})$ | $O(n)$ |

### 4. Suffix Array (后缀数组)

动机：Suffix Tree 理论好，但指针多、cache 不友好、实现复杂；Suffix Array 用一个连续数组更工程化。

定义：把所有 suffix 排序，得到它们的起点数组 SA。例  $T = \text{"inning"}$ ：suffix 起点 0..5；排序后起点为  $SA = [5, 3, 0, 4, 2, 1]$ 。含义： $SA[j] = i$  表示第  $j$  小的后缀是  $T[i..n-1]$ 。

查询：对 SA 做二分。每次比较  $P$  与  $T[SA[mid]..]$  的前缀，最多比  $m$  个字符；二分  $O(\log n)$  次  $\Rightarrow O(m \log n)$ 。

特点：空间小、连续数组、cache friendly，工业界常用。

## 5. $O(n)$ 构造 SA (DC3/Skew, 理解即可)

思想：按  $i \bmod 3$  分组  $B_0/B_1/B_2$ ；先排序  $B_1 \cup B_2$ （占  $2/3$ ），再利用排名信息排序  $B_0$ ，最后 merge。

关键点：先对长度 3 的三元组做 radix sort（固定长度  $\Rightarrow O(n)$ ）；若三元组有重复则递归；比较  $B_0$  时用  $(T[i], R12[i+1])$ ；递推  $T(n) = T(2n/3) + O(n) \Rightarrow O(n)$ 。

## 6. LCP（最长公共前缀）与搜索加速

定义  $\text{lcp}(A, B) = A$  与  $B$  的最长公共前缀长度（如 universe vs university,  $\text{LCP} = 7$ ）。

为什么有用：SA 二分中会反复比较相同前缀；用 LCP 记录“已知相同的前缀长度”，下次比较可以跳过这一段。

结果：搜索从  $O(m \log n)$  可优化到  $O(m + \log n)$ （思想：减少重复字符比较）。

## 7. 这章你要掌握的层级

- Level 1（必须会）：SA 定义、SA 二分查询（ $O(m \log n)$ ）、Suffix Tree vs SA 的对比。
- Level 2（考试重点）：为什么 Suffix Trie 是  $O(n^2)$ ，为什么 Suffix Tree 是  $O(n)$ 。
- Level 3（难点）：DC3 的分组/递归/merge（通常理解思想即可）。

## 8. 总结

把“扫描 text 找 pattern”变成“在后缀索引里查 pattern”。演化路线：Brute Force  $\rightarrow$  Trie  $\rightarrow$  Suffix Trie  $\rightarrow$  Suffix Tree  $\rightarrow$  Suffix Array  $\rightarrow$  LCP 优化。

# Round 8

**Approximate String Comparison (近似字符串比较)**：不再问“P 是否完全出现”，而是问“两个字符串有多像”。应用：diff、拼写纠错、DNA 比对、版本管理等。核心两块：LCS + Edit Distance (编辑距离)。

## 1. Subsequence (子序列) vs Substring (子串)

Subsequence：不要求连续，只要求相对顺序不变。例："programming" 的 subsequence 可以是 "gaming" (按顺序挑出 g-a-m-i-n-g)；"arming" 不是 (顺序对不上)。Substring 必须连续 (本章主要用 subsequence)。

## 2. LCS (Longest Common Subsequence)

定义：两个字符串共同拥有的最长子序列长度 (也可恢复具体序列)。例 A="bird", B="bidder", LCS 长度 3 (如 "bid"/"bir")。

### 2.1 Alignment / Edit Script 直觉

把两串“对齐” (alignment) 能看出共同部分；不在 LCS 里的字符，对应 edit script 里的插入/删除，所以 LCS 与 diff 的思想一致。

## 3. LCS 的 DP (必考)

定义 DP： $L[i][j]$  = LCS(A 的前 i 个字符, B 的前 j 个字符) 的长度 (注意 i, j 从 0..m / 0..n, 空前缀为 0)。

转移：

- 若  $A[i-1] == B[j-1]$ ： $L[i][j] = L[i-1][j-1] + 1$
- 否则： $L[i][j] = \max(L[i-1][j], L[i][j-1])$

Base： $L[0][j] = L[i][0] = 0$

复杂度：时间  $O(mn)$ ，空间  $O(mn)$ 。若只求长度，可用滚动数组把空间降到  $O(\min(m,n))$ 。

### 3.1 回溯 (Backtracking) 怎么找出具体的 LCS (常考手推)

从右下角  $L[m][n]$  开始：

- 若  $A[i-1] == B[j-1]$ ：取该字符，走 
- 否则：看  $L[i-1][j]$  vs  $L[i][j-1]$ ，往更大的方向走 (↑ 或 ←)

直到走到第 0 行/列。

## 4. Edit Distance (编辑距离)

衡量把 A 变成 B 需要多少步操作。最常见：Levenshtein distance (允许 insertion / deletion / substitution, 单位代价 1)。

DP 定义： $D[i][j]$  = A 前 i 个字符变成 B 前 j 个字符的最小代价。

转移 (unit cost)：

- 替换/匹配： $D[i][j] = D[i-1][j-1] + (A[i-1] \neq B[j-1] ? 0 : 1)$
- 插入： $D[i][j] = D[i][j-1] + 1$
- 删除： $D[i][j] = D[i-1][j] + 1$

Base： $D[0][j]=j$ ,  $D[i][0]=i$

复杂度：时间  $O(mn)$ , 空间  $O(mn)$  (同样可滚动数组优化空间)。

## 5. LCS 与 Edit Distance 的关系 (考试爱问)

如果只允许 insertion + deletion (不允许 substitution), 则：

$\text{indel-distance} = m + n - 2 * \text{LCS}(A,B)$

直观：LCS 里的字符“保留”，其余都要删掉/插入。

## 6. 这章你要掌握的 (速记)

- LCS：DP 填表 + 回溯路径 (↖ ↑ ←)
- Edit Distance：DP 三种操作 (替换/插入/删除) + base case
- 会区分：LCS 是 max (求最长), Edit Distance 是 min (求最少操作)